Protocolo Mancomunado

TERCERA ENTREGA: PROYECTO INTEGRADOR

Jason Montenegro, Jafet Araya, Ivan Ríos

Sistema de Archivos

División `del sistema con súper bloque, bitmap, e Inodos.

Múltiple funcionalidad y sin restricciones de tamaños salvo el máximo límite de una partición.

Posee manejo y encapsulación de errores,

```
ez-filesystem REPL commands
──────────────────────────────────────────────────────
format <file> <size_bytes>    Create a new filesystem container
mount <file>                  Mount an existing container
unmount                       Unmount current container
ls <path>                     List directory
cd <path>                     Change directory
pwd                           Print working directory
mkdir <path>                  Create directory
rmdir <path>                  Remove empty directory
touch <path>                  Create empty file
write <path> <text...>        Write text to file (overwrites)
cat <path>                    Print file contents
rm <path>                     Delete file
help                          Show this help
exit                          Exit the REPL
```

Sistema de Archivos

Super block	Bitmap			Inode table		Data blocks		
file_system_identifier total_blocks block_size inode_table_start inode_count data_start blocks_available inodes_available padding[3]	[-][-][-]...[-]	...	[-][-][-]...[-]	Inode_type inode_id total_size created_timestamp direct_blocks[15] indirect_blocks[15] double_indirect_blocks[15] triple_indirect_blocks[15] padding[3]	...	Raw data	...	Raw data

Simulación

- Cliente: construye el mensaje de protocolo correspondiente y espera la respuesta del Fork.

- Intermediario/Fork: descubre servidores disponibles al arrancar, mantiene un mapa de qué figura está en qué servidor.

- Servidor: verifica existencia, envia respuesta o error, auxiliar es LegoRepository para envío de Lista o figura específica.



Simulación

ServerMap: encargado de manejar las conexiones con los servidores, conoce donde recide cada figura → dirección.

Casos de error:
Figura no encontrada en ServerMap, input inválido de usuario, figura borrada.

Bitácoras adaptadas al protocolo compartido: [ENTIDAD] [FECHA_HORA] [TIPO_LOG enrut-info-sol-err] [MENSAJE] + mensaje extra.

- Dirección asociada simulada:

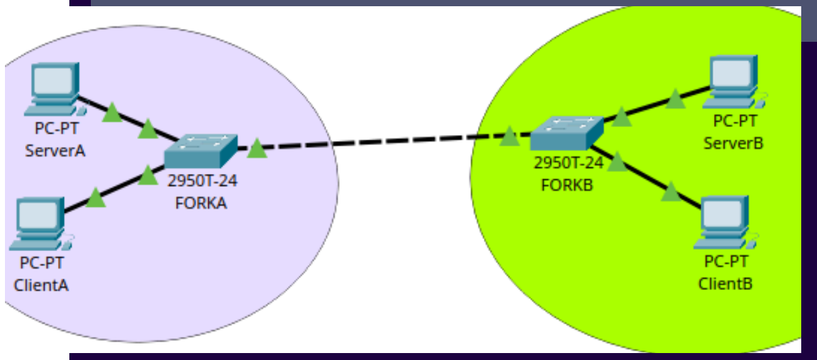
{
"172.16.123.71:8080" -> { "Horse", "lion", "fish", "giraffe" }
}

```
enum class Command : char{
    ACK = 'A',
    CONNECT = 'C',
    DATA = 'D',
    FORK= 'F',
    GET = 'G',
    KILL = 'K',
    PATCH = 'P',
    QUIT = 'Q',
    REQUEST = 'R',
    SERVER = 'S',
    UNKNOWN = '?'
};
```



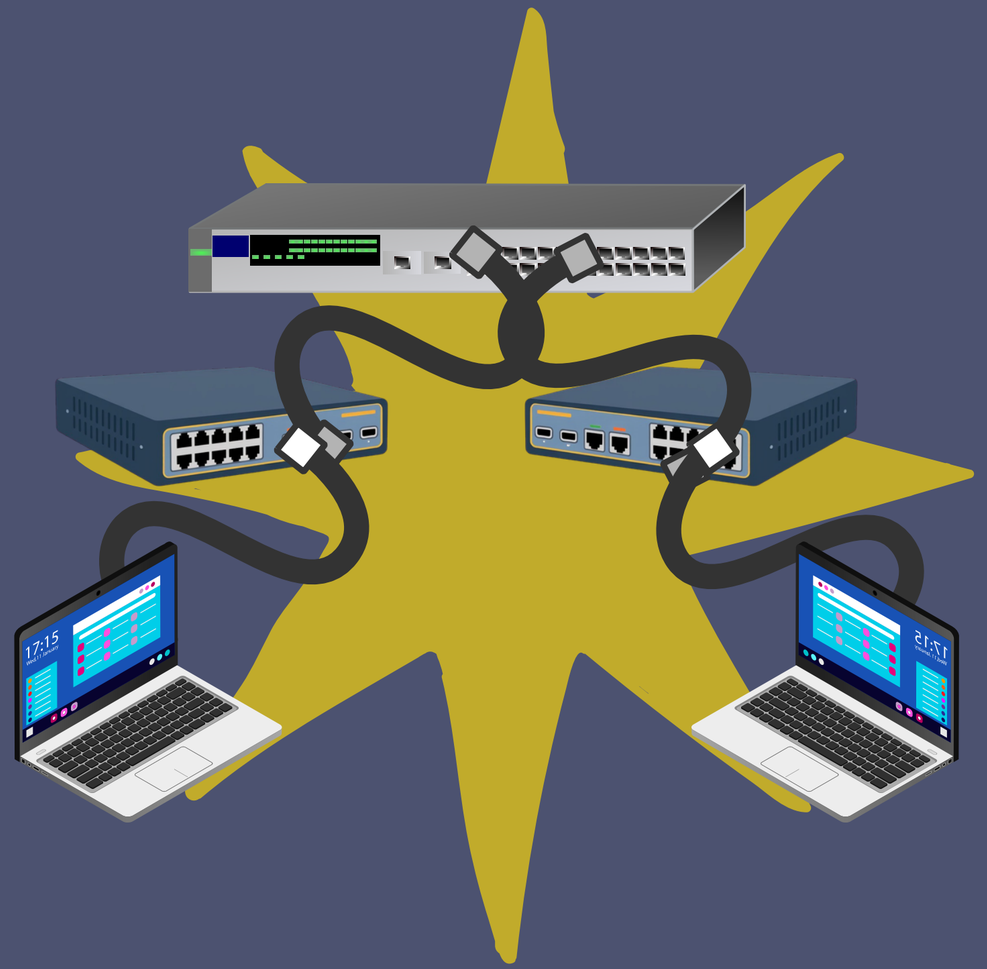
Ciclo esperado

- **Cliente solicita figura:** "giraffe", segmento 1 Client → P/G/giraffe&1 → Fork.
- Fork A **consulta** su tabla global (ServerMap) "172.16.123.82" → { fish, giraffe }.
- De no estar Fork A → P/D/404:giraffe → Client.
- Si no esta en el servidor actual entonces se encarga de redireccionar con broadcast a otro Fork B que si posea la pieza, lo busca en su serverMap.
    - Si no está en su server el último recurso es actualizar su serverMap.
- Fork B responde a Fork A y Fork A retorna solicitud al cliente.



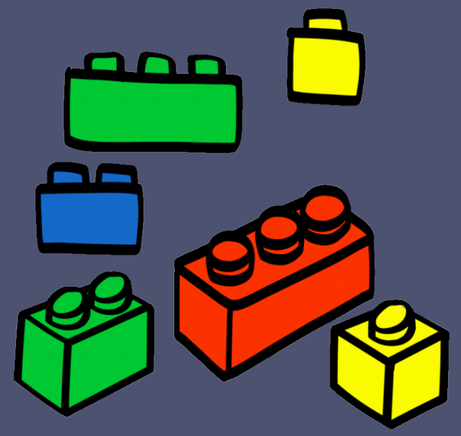
L2 y L3 Configuración

- Asignación de VLAN 260 de laboratorio, con puertos troncales.
- Isla 6: IP 172.16.123.96/28, IP L3 .97, IP L2 .98.
- L2 con puerto troncal Gig0/1 y Pools dhcp [99-110] .
- enemigo: conexión a wifi (╥﹏╥).



¡LEGO SERVER!

SOLICITUDES HTTP/HTTPS



SERVIDOR CON MISMA LÓGICA

El cliente nachos parsea la respuesta

Ya tiene todos los requisitos nuestro servidor.

Solo habla http el server

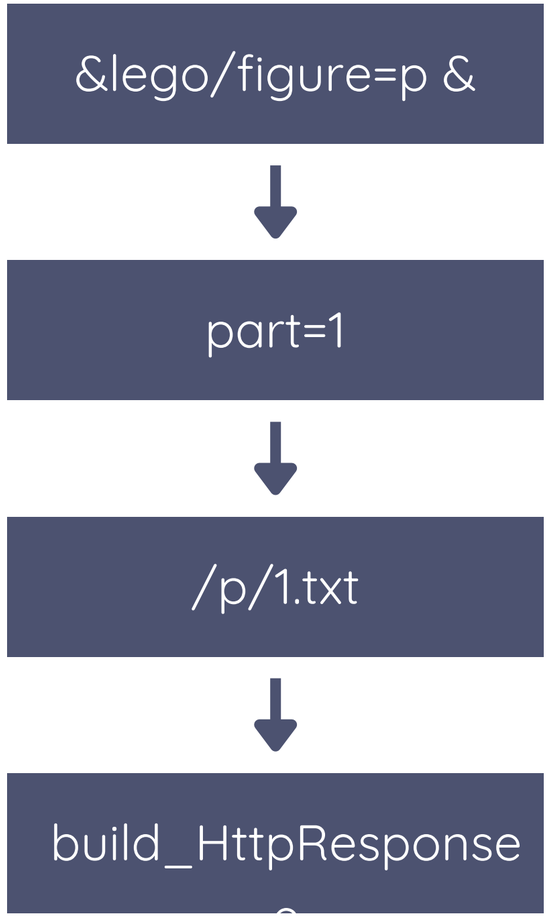
Más feliz

MISMA LÓGICA DE SOLICITUD

Recibe y parsea la solicitud, si es Request verifica que tenga Get al incio, despues busca la solicitud.

extrae del header , la figura y el numero de figura (en caso de que venga list/figures devuelve lista de figuras)

Crea la ruta en el file system con base al nombre de la figura y la parte .





CLIENTE NACHOS

- Parseo de contenido Http
- Usando sockets de linux de fondo
- LLamados al sistema implementados en exeption
- Tabla Amplia de archivos

SALIDA ESPERADA

```
Cleaning up...
ivan@LAPTOP-OL933GT8:~/proyectos/Nachos/code/userprog$ ./nachos -x ../test/socket

Request construido: GET &lego/figure=duck&part=1 HTTP/1.0
User-Agent: nachos



se viene chavales
Cantidad total de piezas:5
brick 2x2 slope orange: 1
brick 2x2 yellow: 2
brick 2x3 yellow: 2

No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 5187, idle 0, system 10, user 5177
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...
ivan@LAPTOP-OL933GT8:~/proyectos/Nachos/code/userprog$
```

¡Presentación de funcionalidades!

